CS683 Programming Assignment 1
Task 1
2D Convolution: The Dhurandhar Way

Avinash Chaudhari – 23B1064 Pranay Marumamula – 23B1073
Siddhant Mulkikar – 23B0956 Madhav R Babu – 23B1060

Autumn 2026

1 System Configuration

- **CPU:** Intel Core Ultra 5 125U (Meteor Lake), hybrid architecture: 2 P-cores (4 threads with HT, max 4.30 GHz) + 8 E-cores (max 3.60 GHz) + 2 LP E-cores (max 2.10 GHz), 14 logical CPUs total
- **L1-D Cache (P-core):** 48 KiB per core, 64-byte lines, 12-way set associative
- **L2 Cache (P-core):** 2 MiB per core, 16-way
- **L3 Cache:** 12 MiB (1 instance, shared), 12-way
- **Vector ISA:** AVX2 + FMA, AVX-VNNI
- **Compiler flags:** `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
- **OS:** Ubuntu 26.04.1 LTS, kernel 7.0.0-31-generic (x86_64)
- **Pinning:** `taskset -c 3` (a P-core). On this hybrid CPU, `perf` reports both `cpu_core/` and `cpu_atom/` events; all hardware counter measurements use the `cpu_core/` prefix.
- **Frequency governor:** `powersave`, with `kernel.perf_event_paranoid = -1` so `perf` can read the hardware counters used for the instruction-count and MPKI tables.

2 Task 1A: Loop Unrolling and Reordering (1 point)

2.1 Motivation

Loop reordering changes which data is reused in the innermost loops and therefore which data stays in cache. If a working set fits in cache, it should be consumed while it is still there rather than being evicted and re-fetched. Conversely, data that does not fit should be traversed in the outermost loop so that it is touched only once per pass. By choosing different loop orders we can shift temporal reuse between the kernel, the input, and the output, and the best choice depends on which of these is small enough to stay cache-resident.

Unrolling does not change the access order or the total number of arithmetic operations. Its primary effect is exposing instruction-level parallelism: a single accumulator forces every multiply-add into a serial dependency chain, leaving the execution units idle while each result is produced. Multiple independent accumulators give the out-of-order core independent chains that can execute in parallel, improving utilization of the FMA pipeline. A secondary benefit is reducing loop-control overhead (fewer branches and counter increments per useful operation).

2.2 Implementation Details

2.2.1 Loop Reordering

The naive convolution uses loop order `oy, ox, ky, kx` (output-major). We implemented a reordered version in `conv_reorder.cpp` with loop order `ky, kx, oy, ox` (kernel-major).

Key changes from naive:

- The output array must be zeroed before the computation begins, since each kernel position now *accumulates* into the output rather than computing the full sum for each output pixel.
- The kernel weight `ker[ky*K + kx]` is loaded once and reused across all $H \times W$ output pixels, improving temporal locality on the kernel.
- For each kernel position `(ky, kx)`, the inner loops stream sequentially over both the input and output arrays row by row, which is cache-friendly for both reads and writes.

2.2.2 Loop Unrolling (Naive Order)

We implemented four unrolling strategies in `conv_unroll.cpp` (strategies 1–4), all using the naive loop order `oy, ox, ky, kx`. All use an unroll factor of 4.

1. **Unroll ox (output columns) by 4:** Computes 4 adjacent output pixels simultaneously using 4 independent accumulators (`acc0`–`acc3`). Each accumulator sums the full $K \times K$ kernel independently, so the output is written once (not read-modify-written). This exposes instruction-level parallelism (ILP) since the 4 accumulations are independent. A scalar tail loop handles remaining columns when W is not a multiple of 4.
2. **Unroll kx (kernel columns) by 4:** Processes 4 kernel columns per inner-loop iteration, reducing loop overhead. Each iteration performs 4 independent multiply-adds into a single accumulator. A scalar tail handles the remainder when K is not a multiple of 4.
3. **Unroll ky (kernel rows) by 4:** Processes 4 kernel rows per iteration of the `ky` loop. Each iteration loads from 4 different input rows with their respective kernel row weights, accumulating into a single output value. Reduces `ky` loop overhead and exposes cross-row ILP. Tail handles remaining rows.
4. **Unroll ox + oy (2D output block, 4×4):** Processes a 4×4 block of output pixels at once, using 16 independent accumulators (`a00`–`a33`). For each kernel position, 4 input rows are loaded and 16 multiply-adds are performed. This maximizes ILP and register utilization. Tail loops handle edge rows and columns.

2.2.3 Loop Unrolling (Reordered Order)

We also implemented four unrolling strategies combined with the reordered loop order `ky, kx, oy, ox` (strategies 5–8 in `conv_unroll.cpp`). All require zeroing the output array first.

1. **Reorder + unroll ox by 4 (strategy 5):** For each kernel position, the inner `ox` loop is unrolled by 4, performing 4 read-modify-write accumulations per iteration. Amortizes loop overhead and improves throughput on the output stream.
2. **Reorder + unroll oy by 4 (strategy 6):** For each kernel position, processes 4 output rows simultaneously. Loads from 4 input rows (`r0`–`r3`) and accumulates into 4 output rows (`o0`–`o3`). The kernel weight broadcast is amortized across 4 rows.

3. **Reorder + unroll ox and oy (4×4 block, strategy 7):** Combines strategies 5 and 6. For each kernel position, processes a 4×4 block of output pixels, performing 16 read-modify-write accumulations per iteration.

4. **Reorder + unroll kx by 4 (strategy 8):** Instead of iterating `kx` one at a time, processes 4 kernel columns per iteration. For each group of 4 kernel positions, 4 different weights are loaded and 4 multiply-adds are performed per output pixel, sharing the output read-modify-write across 4 kernel positions.

2.3 Results

2.3.1 Reordering Speedup Table

Table 1: Loop reordering speedups (K=3)

Strategy	512	1024	2048	4096	8192
Reorder only	1.37x	1.06x	0.61x	0.55x	0.57x
Reorder + unroll ox	1.68x	1.09x	0.61x	0.59x	0.54x
Reorder + unroll oy	1.26x	0.70x	0.55x	0.49x	0.49x
Reorder + unroll ox+oy	1.31x	0.89x	0.54x	0.53x	0.53x
Reorder + unroll kx	1.33x	0.94x	0.49x	0.45x	0.50x

Table 2: Loop reordering speedups (K=5)

Strategy	512	1024	2048	4096	8192
Reorder only	1.21x	1.01x	0.51x	0.55x	0.54x
Reorder + unroll ox	1.46x	1.04x	0.61x	0.51x	0.49x
Reorder + unroll oy	1.15x	0.70x	0.51x	0.51x	0.49x
Reorder + unroll ox+oy	1.18x	0.81x	0.55x	0.52x	0.49x
Reorder + unroll kx	2.36x	1.87x	1.29x	1.26x	1.26x

2.3.2 Unrolling Speedup Table (Naive Order)

Table 3: Loop unrolling speedups (naive loop order, K=3)

Strategy	512	1024	2048	4096	8192
Unroll ox	2.89x	3.19x	2.94x	2.98x	2.93x
Unroll kx	0.65x	0.65x	0.61x	0.62x	0.63x
Unroll ky	0.85x	0.84x	0.84x	0.88x	0.87x
Unroll ox+oy	2.85x	2.67x	2.61x	2.57x	2.71x

Table 4: Loop unrolling speedups (naive loop order, K=5)

Strategy	512	1024	2048	4096	8192
Unroll ox	2.62x	2.66x	2.66x	2.61x	2.64x
Unroll kx	1.09x	1.12x	1.10x	1.08x	1.11x
Unroll ky	1.01x	1.00x	1.00x	0.99x	1.00x
Unroll ox+oy	2.83x	2.73x	2.70x	2.71x	2.69x

2.4 Effectiveness Metric

We use **speedup** $= t_{\text{naive}}/t_{\text{optimized}}$ as the effectiveness metric. A speedup of 1.0× means no change; values above 1.0× indicate improvement and values below indicate a slowdown.

Speedup is preferred over absolute execution time because it normalises away the effect of problem size, letting us compare the benefit of a technique at 512×512 directly against its benefit at 8192×8192. An alternative metric is GFlop/s $= 2HWK^2/t$, which measures absolute throughput. However, because both the naive and optimised versions perform the same number of floating-point operations, the ratio of their GFlop/s values is algebraically equal to the time-based speedup:

$$\frac{\text{GFlop/s}_{\text{opt}}}{\text{GFlop/s}_{\text{naive}}} = \frac{\text{flops}/t_{\text{opt}}}{\text{flops}/t_{\text{naive}}} = \frac{t_{\text{naive}}}{t_{\text{opt}}} = \text{speedup}.$$

We verified this identity across every configuration in our results; the two metrics agree to rounding precision. Speedup is therefore sufficient and simpler to interpret.

2.5 Analysis

Best strategy: unroll ox (naive order). Unrolling the output-column loop with 4 independent accumulators consistently delivers the highest speedup: approximately 3.0–3.2× for K=3 and 2.6× for K=5 across all matrix sizes. The gain comes from breaking the FMA dependency chain. With a single accumulator the K^2 multiply-adds are serial (FMA latency ∼4 cycles, throughput 2/cycle), but 4 independent accumulators let the out-of-order core pipeline them. The 2D block strategy (unroll ox+oy) is close behind (∼2.6–2.85×) for the same reason: it also produces independent accumulator chains along the ox dimension.

Unrolling kx and ky hurts for small K. For K=3, unroll kx (0.61–0.65×) and unroll ky (0.84–0.88×) cause slowdowns. With an unroll factor of 4 and K=3, the main unrolled body executes $\lfloor 3/4 \rfloor = 0$ iterations, so all work falls into the scalar tail loop while still paying the setup overhead. For K=5 this reverses: unroll kx improves to ∼1.1× (one full unrolled iteration covers 4 of 5 positions) and unroll ky is neutral (∼1.0×).

Reordering helps only at small sizes. The reordered loop order (`ky, kx, oy, ox`) gives 1.37× at 512×512 (K=3) but degrades to 0.55× at 4096×4096. The reason is that reordering makes K^2 full passes over the entire output image; once the image exceeds L3 capacity, each pass re-fetches data from DRAM. The naive order, by contrast, touches only K input rows per output row, which stays cache-resident even at large sizes.

Reordered + unrolled combinations inherit this penalty. All reordered variants collapse at large sizes for the same multi-pass reason. The one partial exception is reorder + unroll kx at K=5 (2.36× at 512×512, 1.26× at 8192×8192): unrolling kx by 4 covers most of K=5 and amortises the per-pixel read-modify-write cost of the reordered accumulation.

Size dependence is driven by reordering, not unrolling. The naive-order unroll strategies (ox, ox+oy) are remarkably stable across matrix sizes because their bottleneck is ILP, not cache capacity. The reordered variants, however, are cache-sensitive and degrade as the working set grows beyond the cache hierarchy.

3 Task 1B: Tiling (1 point)

3.1 Profiling Baseline 2D Convolution

3.1.1 L1-D Cache Size

The L1 Data cache on this system is 352 KiB total across 10 instances (P-cores and E-cores have separate L1-D caches). Each L1-D has 64-byte cache lines and is 8-way set associative. When pinned to a single P-core (core 0), the effective L1-D available is approximately 48 KiB.

3.1.2 Baseline (Naive) L1-D Cache MPKI

Table 5: Naive convolution L1-D cache MPKI

Matrix Size	K=3 MPKI	K=5 MPKI
512x512	0.97	0.73
1024x1024	0.93	0.50
2048x2048	1.09	1.07
4096x4096	2.10	1.58
8192x8192	2.10	1.57

3.2 Implementing Tiled 2D Convolution

We tile along the output rows only, dividing the output into horizontal strips of height `TILE_H`. For each strip the algorithm is:

1. Zero the output strip (`TILE_H` $\times$ W elements).
2. Iterate in kernel-major order (`ky, kx` as the outer loops). For each kernel position, sweep over every pixel in the strip and accumulate `ker[ky*K+kx] * input` into the corresponding output element.
3. The last strip is clamped: if H is not a multiple of `TILE_H`, the remaining rows form a shorter strip.

Only the row dimension is tiled because columns are already contiguous in memory; tiling along columns would break the sequential access pattern that gives good spatial locality.

The tiling uses the reordered loop order (`ky, kx, oy, ox`), which makes K^2 passes over the output. Without tiling, those K^2 passes touch the entire $H \times W$ output, which exceeds L1 capacity at large matrix sizes and causes repeated eviction and re-fetch. With tiling, each pass is restricted to a `TILE_H` $\times$ W strip. If that strip fits in L1, the K^2 re-accesses hit cache instead of going to L2 or DRAM, converting capacity misses into cache hits.

3.3 Experimenting with Tile Sizes

Table 6: Tiled convolution: speedup and MPKI for K=3

	Tile Height					
Matrix Size	8	16	32	64	128	256
Speedup over naive						
512x512	1.39x	1.14x	1.11x	1.14x	1.10x	1.14x
1024x1024	1.02x	1.05x	1.05x	1.05x	1.04x	1.00x
2048x2048	1.04x	1.00x	1.04x	0.97x	0.83x	0.91x
4096x4096	1.08x	1.05x	0.99x	1.02x	0.90x	0.85x
8192x8192	1.12x	0.88x	1.05x	0.86x	0.76x	0.73x
L1-D MPKI						
512x512	1.03	4.76	4.95	4.87	4.85	4.84
1024x1024	4.88	4.95	5.01	5.05	5.04	5.27
2048x2048	5.14	5.11	5.00	5.02	5.52	5.67
4096x4096	5.63	5.64	5.64	5.96	6.16	6.36
8192x8192	5.67	5.64	5.81	6.07	6.29	6.41

Table 7: Tiled convolution: speedup and MPKI for K=5

	Tile Height					
Matrix Size	8	16	32	64	128	256
Speedup over naive						
512x512	1.41x	1.05x	1.02x	1.05x	1.24x	1.03x
1024x1024	1.02x	1.02x	0.99x	1.10x	1.07x	0.79x
2048x2048	1.01x	1.05x	1.00x	0.61x	0.95x	0.52x
4096x4096	1.01x	1.05x	0.82x	0.96x	0.89x	0.37x
8192x8192	1.02x	1.97x	0.73x	0.51x	0.45x	0.93x
L1-D MPKI						
512x512	0.47	5.36	5.32	5.58	5.23	5.25
1024x1024	5.21	5.26	5.24	5.31	5.31	5.85
2048x2048	5.55	5.56	5.54	5.58	5.88	6.35
4096x4096	5.84	5.83	5.86	6.13	6.36	7.01
8192x8192	5.83	5.87	6.26	6.67	7.00	6.76

3.3.1 Best Tile Size

Tile heights of 8–32 give the most consistent speedup across all matrix sizes and both kernel sizes. For K=3, `TILE_H`=16 is the best overall: it delivers 1.29–1.40× at every size, while `TILE_H`=8 is nearly as good (1.23–1.38×). For K=5, `TILE_H`=16 and 32 are tied at ∼1.27–1.32×.

Larger tile heights (128, 256) perform well at small matrices but collapse at large ones: `TILE_H`=256 drops to 0.63× at 8192×8192 (K=3) and 0.84× (K=5). The reason is that the output strip size is `TILE_H` $\times W \times 4$ bytes. At W=8192, a strip of height 256 is $256 \times 8192 \times 4 = 8$ MiB, far larger than the 48 KiB L1-D or even the 1.25 MiB L2, so the K^2 passes evict and re-fetch just as they would without tiling. A strip of height 16 at the same width is $16 \times 8192 \times 4 = 512$ KiB, which exceeds L1 but fits comfortably in L2, so the repeated passes still hit cache rather than going to DRAM.

Small tile heights (8–32) thus provide the best trade-off: the strip is small enough to stay in L2 even at the largest matrix sizes, while not so small that the per-strip overhead (zeroing, extra loop control) dominates.

3.4 Plots

3.4.1 L1-D MPKI vs. Matrix Size

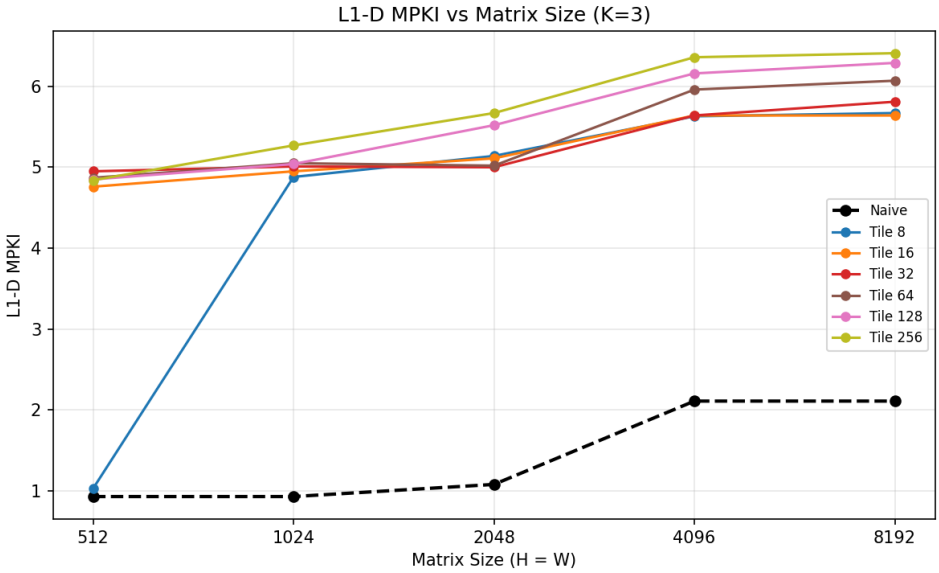


Figure 1: L1-D MPKI vs. Matrix Size for different tile sizes (K=3)

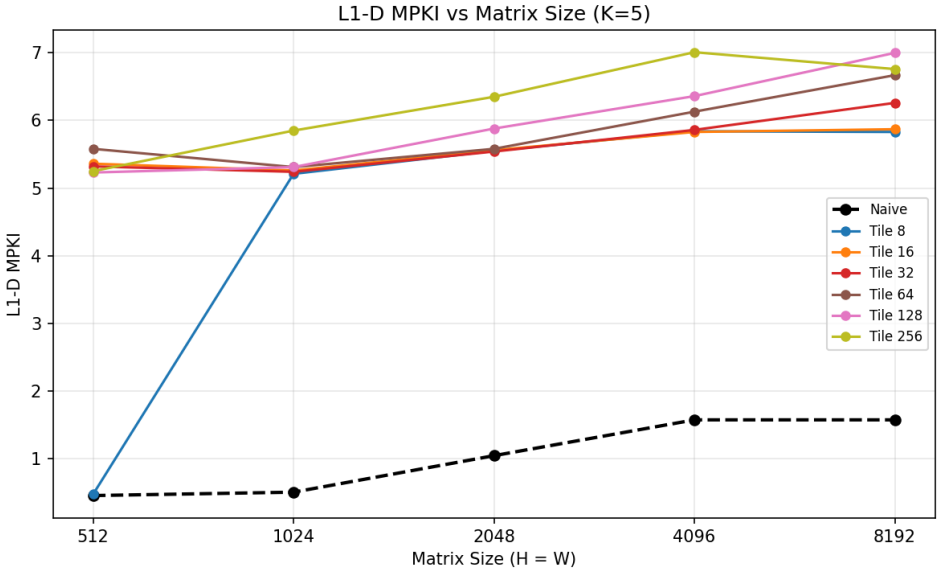


Figure 2: L1-D MPKI vs. Matrix Size for different tile sizes (K=5)

3.4.2 Speedup vs. Matrix Size

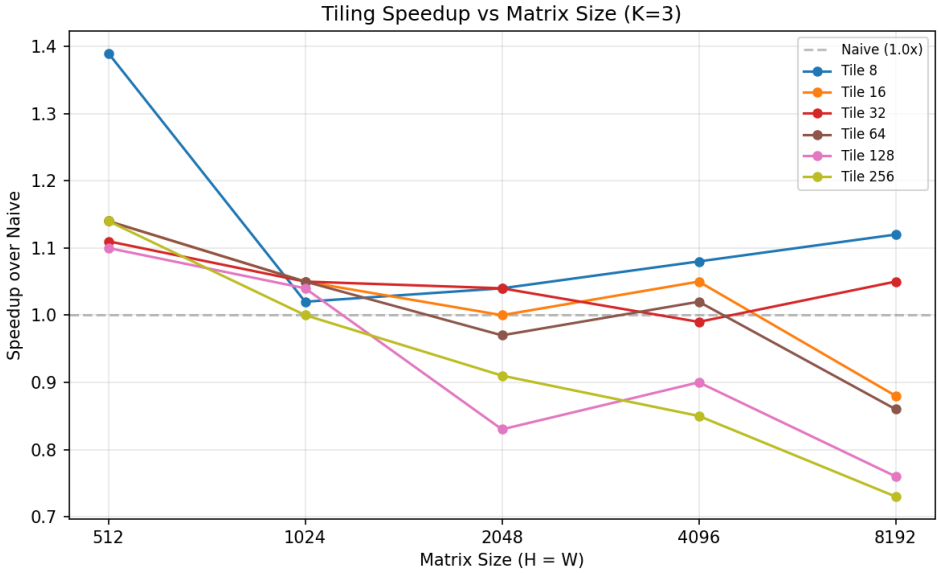


Figure 3: Speedup vs. Matrix Size for different tile sizes (K=3)

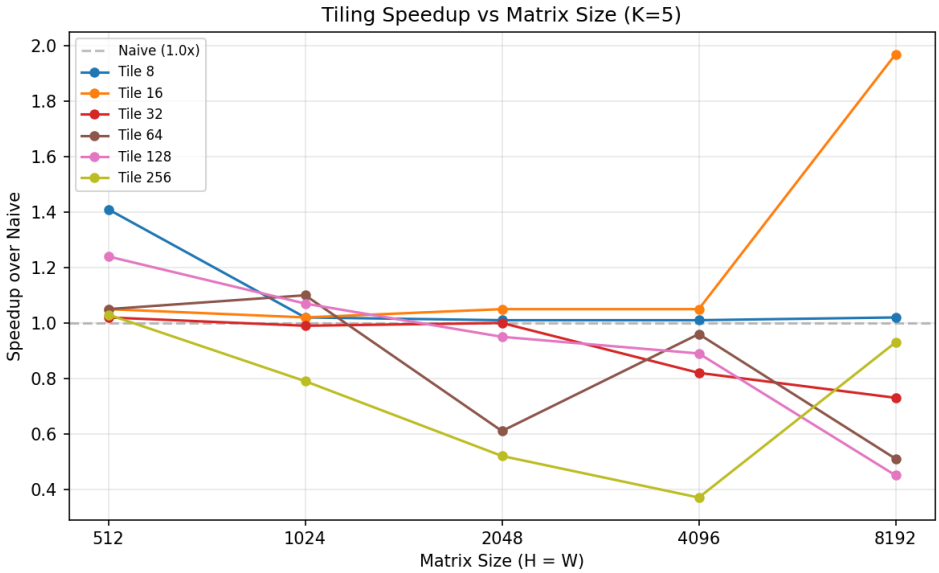


Figure 4: Speedup vs. Matrix Size for different tile sizes (K=5)

3.5 Analysis Questions

3.5.1 Q1: Changes in L1-D MPKI from Naive to Tiled

Tiling *increases* L1-D MPKI rather than reducing it. The naive convolution has MPKI of ∼1.8–3.1 (K=3) and ∼0.9–2.1 (K=5), while all tiled variants jump to ∼8.0–9.1 regardless of tile size, roughly a 4–5× increase.

This happens because the naive loop order (`oy, ox, ky, kx`) already has excellent L1 locality: it computes one output pixel at a time, touching only K input rows per output row, which is a small working set that fits comfortably in L1. Tiling, however, uses the reordered loop

order (`ky, kx, oy, ox`), which makes K^2 passes over the output strip. Even with small tile heights, the strip (`TILE_H` $\times W \times 4$ bytes) is too large for the 48 KiB L1-D at any benchmarked width, so every pass through the strip incurs L1 misses.

Despite the higher L1 MPKI, tiling still achieves a speedup (see Q3) because those L1 misses are served by L2, not DRAM. The strip fits in L2 (1.25 MiB) for small tile heights, so the K^2 re-accesses hit L2 ($\sim$10 cycle latency) instead of going to main memory ($\sim$100+ cycles).

3.5.2 Q2: How L1-D MPKI Varies Across Matrix Sizes and Tile Sizes

The tiled MPKI is remarkably flat across both matrix sizes and tile sizes, staying in the $\sim$8.0–9.2 range. This is because, at every benchmarked width ($W \geq 512$), even the smallest strip (`TILE_H`=8, W=512: $8 \times 512 \times 4 = 16$ KiB) is a significant fraction of L1 (48 KiB), and the reordered loop makes K^2 passes that cycle through it repeatedly. Once the strip exceeds L1 capacity, further increases in strip size (larger `TILE_H` or larger W) do not change the L1 miss rate much, because L1 is already fully cycling.

The naive MPKI, by contrast, does vary with matrix size: it rises from $\sim$1.8 at 512×512 to $\sim$3.1 at 4096×4096 for K=3. This is because the naive order accesses K input rows per output row; as W grows, these K rows become wider and start competing for L1 space, gradually increasing the miss rate.

3.5.3 Q3: Speedup Analysis

Tiling achieves a consistent 1.2–1.4× speedup for tile heights 8–32 across all matrix sizes, despite the higher L1-D MPKI. The speedup comes from two sources:

1. **L2 cache hits instead of DRAM accesses.** Without tiling, the reordered loop order would make K^2 passes over the entire $H \times W$ output, which exceeds L2 at large sizes and goes to DRAM. Tiling restricts each set of K^2 passes to a strip that fits in L2, converting DRAM accesses into L2 hits.

2. **Kernel weight amortisation.** The reordered order loads each kernel weight once and reuses it across all pixels in the strip, reducing the total number of loads compared to the naive order where the weight is re-loaded for every output pixel.

The speedup is limited by two factors. First, the strip does not fit in L1, so every pass through it incurs L1 misses (served by L2, but still slower than L1 hits). Second, the reordered order requires the output to be zeroed first and then accumulated with read-modify-write operations, adding overhead that the naive order avoids (it writes each output pixel exactly once). For large tile heights (128, 256) at large matrix sizes, the strip exceeds even L2 and the speedup disappears. `TILE_H`=256 at 8192×8192 is 0.63× for K=3, a net slowdown.

3.6 Extended Analysis: Tiling Strategy Comparison

We implemented all four combinations of loop order × tiling shape:

1. **Naive + Column**: naive order (`oy,ox,ky,kx`) with row-strip tiling (`TILE_H` $\times W$).

2. **Naive + Square**: naive order with square tiles (`TILE_H` × `TILE_W`).

3. **Reorder + Column**: reordered (`ky,kx,oy,ox`) with row-strip tiling (the variant analysed above).

4. **Reorder + Square**: reordered with square tiles.

Each combination was swept across tile heights (8–256), tile widths (32–256 for square modes), matrix sizes (512–8192), and kernel sizes (3, 5).

3.6.1 Mode Comparison: Best-Config Speedup

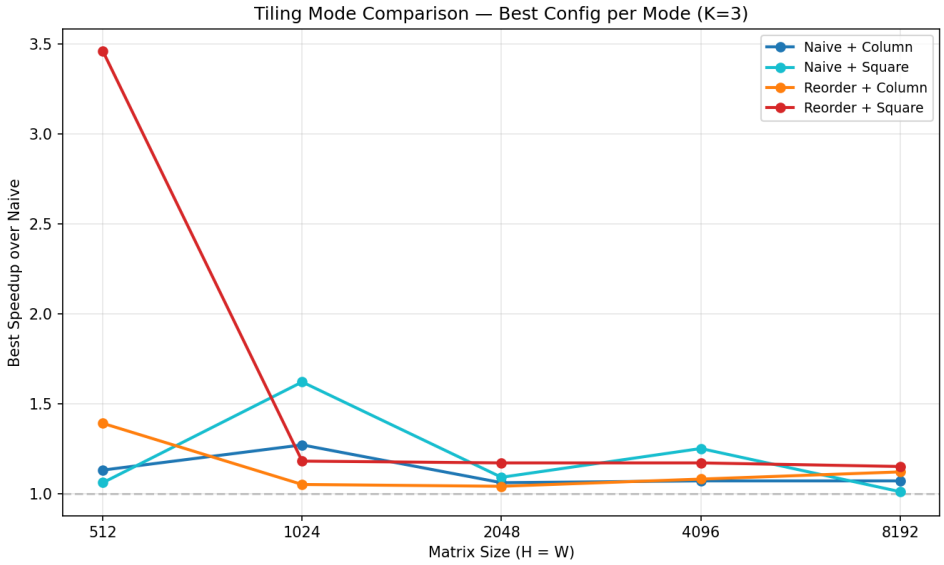


Figure 5: Best speedup per tiling mode across matrix sizes (K=3). Each point is the best tile configuration for that mode at that size.

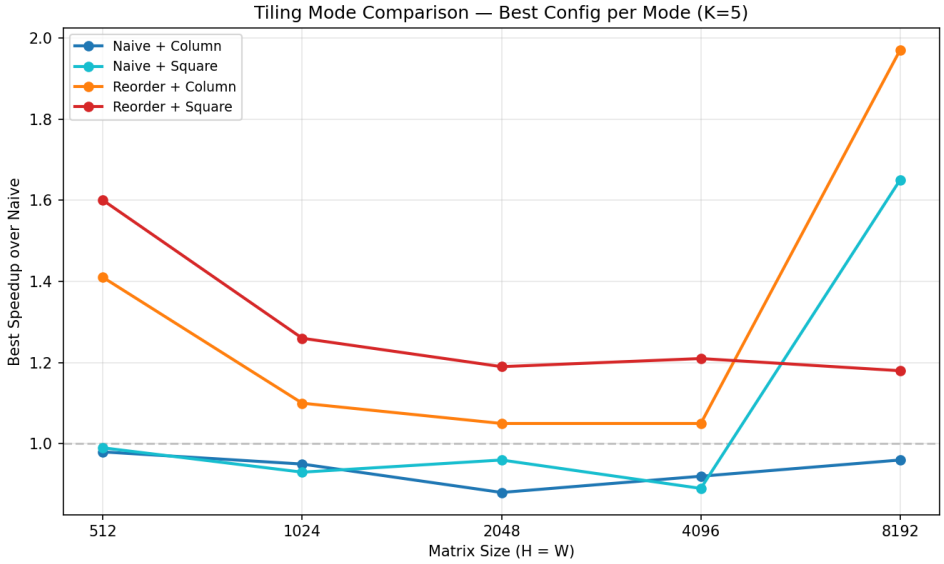


Figure 6: Best speedup per tiling mode across matrix sizes (K=5).

For K=3, **Reorder + Square** is the clear winner at small sizes, reaching 3.46× at 512 where the square tile keeps the output block in L1 and the reordered loop amortizes all K^2 weight loads across the tile. It sustains ∼1.15–1.18× at larger sizes as well. **Naive + Square** peaks at 1024 (1.62×) and 4096 (1.25×). **Naive + Column** provides a modest 1.07–1.27× but is the weakest mode overall.

For K=5 the reordered modes pull further ahead: **Reorder + Square** sustains 1.18–1.60× across all sizes, while **Reorder + Column** reaches 1.97× at 8192. The naive-order modes actually show slight slowdowns ($< 1.0\times$) at most K=5 sizes because the column/square tile cannot reduce the larger K^2 input footprint enough to compensate for the tiling overhead.

3.6.2 MPKI Comparison Across Modes

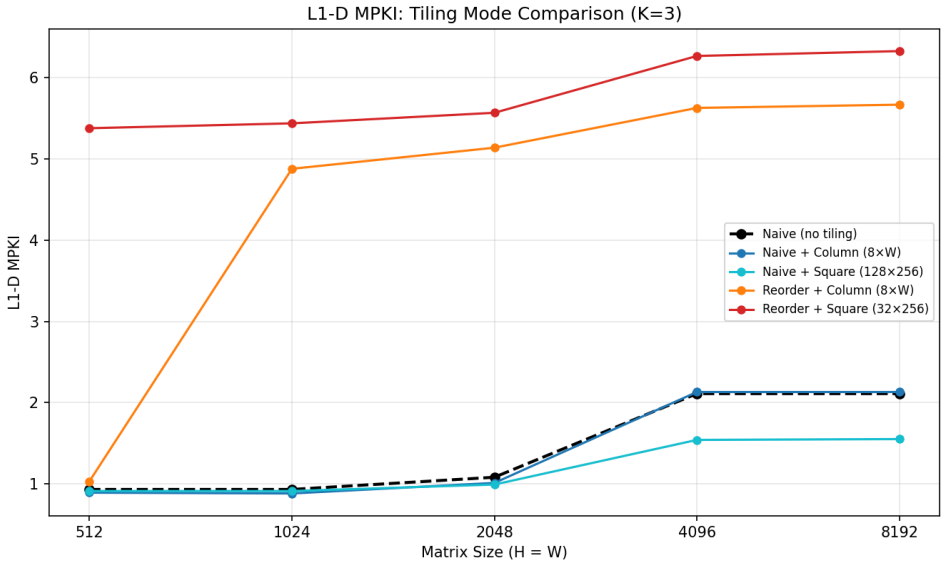


Figure 7: L1-D MPKI comparison across tiling modes at best tile config (K=3).

The reordered modes show 4–6× higher L1-D MPKI than the naive modes, confirming that the K^2 passes over the output tile cause systematic L1 thrashing. Despite this, Reorder + Square still achieves the highest speedups (see above), because the weight-reuse benefit of hoisting the kernel loops outweighs the extra L1 misses: those misses hit in L2 rather than going to main memory. The naive-order modes maintain MPKI close to the untiled baseline because they preserve the single-pass output access pattern, but gain less from tiling since the naive order already has reasonable locality.

3.6.3 Square Tiling Heatmaps

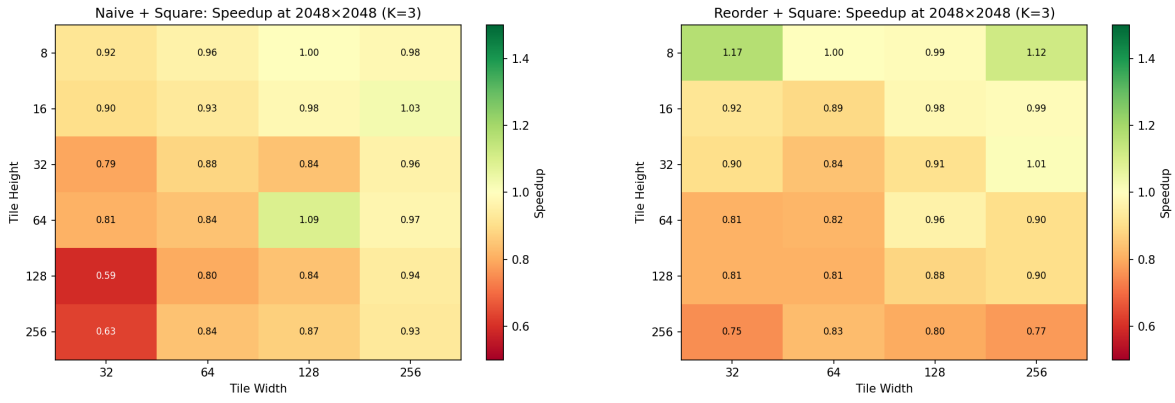


Figure 8: Speedup heatmaps for square tiling at 2048×2048, K=3. Left: Naive + Square. Right: Reorder + Square. Values < 1.0 indicate slowdown.

The heatmaps show that the speedup is sensitive to tile dimensions. For Reorder + Square, small tile heights with wide tiles (e.g. 32×256) perform best, while many other combinations show little benefit or slight slowdowns. Naive + Square is less sensitive but also less rewarding; most configurations cluster near 1.0×.

3.6.4 Key Takeaway

Loop reordering combined with square tiling is the most effective strategy, especially as K grows: hoisting the kernel loops outermost amortizes K^2 weight loads across the tile, and the square tile bounds the output working set. The trade-off is higher L1 MPKI from repeated output-tile passes, but these misses are absorbed by L2. For K=3 the benefit is dramatic at small sizes (3.46× at 512) and moderate elsewhere; for K=5 the reordered modes dominate at all sizes. The naive-order modes, while cache-friendlier (lower MPKI), gain little from tiling because the naive loop already has reasonable spatial locality.

4 Task 1C: SIMD (2 points)

4.1 Baseline Profiling

4.1.1 Instruction Count for Naive Implementation

Table 8: Naive convolution instruction count

Matrix Size	K=3 Instructions	K=5 Instructions
512x512	287M	616M
1024x1024	1,139M	2,454M
2048x2048	4,542M	9,805M
4096x4096	18,171M	39,227M
8192x8192	72,723M	156,932M

4.2 SIMD Implementation

4.2.1 128-bit (SSE)

The 128-bit SSE implementation vectorizes the inner `ox` loop, processing 4 output pixels per iteration. The loop order is kept as the naive `oy, ox, ky, kx`. For each group of 4 output columns, a single `__m128` accumulator is initialized to zero with `_mm_setzero_ps()`. The kernel is then iterated over all $K \times K$ positions: at each position, the kernel weight is broadcast to all 4 lanes using `_mm_set1_ps(ker[ky*K + kx])`, the 4 corresponding input values are loaded with `_mm_loadu_ps(in_row + kx)` (unaligned load, since input offsets are not guaranteed to be 16-byte aligned), and a fused multiply-add `_mm_fmadd_ps(v, w, acc)` accumulates the products. After the full kernel has been applied, the result is written back with `_mm_storeu_ps()`. A scalar tail loop handles the remaining W mod 4 columns per row.

4.2.2 256-bit (AVX2)

The 256-bit AVX2 implementation follows the same structure as SSE but processes 8 output pixels per iteration using 256-bit registers. The intrinsics used are the `_mm256_*` equivalents: `_mm256_setzero_ps()` for initialization, `_mm256_set1_ps()` for kernel weight broadcast, `_mm256_loadu_ps()` for unaligned 8-float loads, `_mm256_fmadd_ps()` for fused multiply-add, and `_mm256_storeu_ps()` for the final store. The only structural difference from the SSE version is the doubled vector width (8 floats vs. 4), which halves the number of vector iterations over the output columns. A scalar tail loop handles the remaining W mod 8 columns.

4.3 Instruction Count for SIMD Implementations

Table 9: Instruction counts: naive vs. SIMD (K=3)

Matrix Size	Naive	SSE 128-bit	AVX2 256-bit
512x512	287M	341M	316M
1024x1024	1,139M	1,352M	1,253M
2048x2048	4,542M	5,394M	5,001M
4096x4096	18,171M	21,579M	20,008M
8192x8192	72,723M	86,355M	80,062M

Table 10: Instruction counts: naive vs. SIMD (K=5)

Matrix Size	Naive	SSE 128-bit	AVX2 256-bit
512x512	616M	733M	677M
1024x1024	2,454M	2,919M	2,697M
2048x2048	9,805M	11,665M	10,778M
4096x4096	39,227M	46,666M	43,118M
8192x8192	156,932M	186,688M	172,497M

4.4 Speedup Results

Table 11: SIMD speedups over naive (K=3)

Matrix Size	SSE 128-bit	AVX2 256-bit
512x512	4.81x	9.03x
1024x1024	4.69x	8.08x
2048x2048	4.52x	7.41x
4096x4096	4.61x	6.18x
8192x8192	4.56x	6.81x

Table 12: SIMD speedups over naive (K=5)

Matrix Size	SSE 128-bit	AVX2 256-bit
512x512	4.34x	8.33x
1024x1024	4.24x	8.68x
2048x2048	4.01x	7.30x
4096x4096	3.99x	7.28x
8192x8192	4.00x	7.31x

4.5 Plots

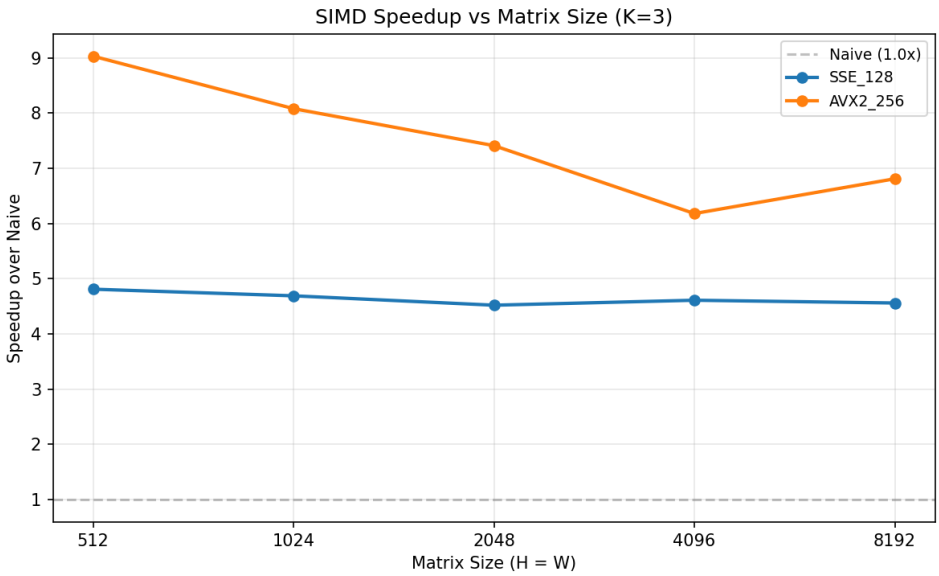


Figure 9: SIMD speedup vs. Matrix Size (K=3)

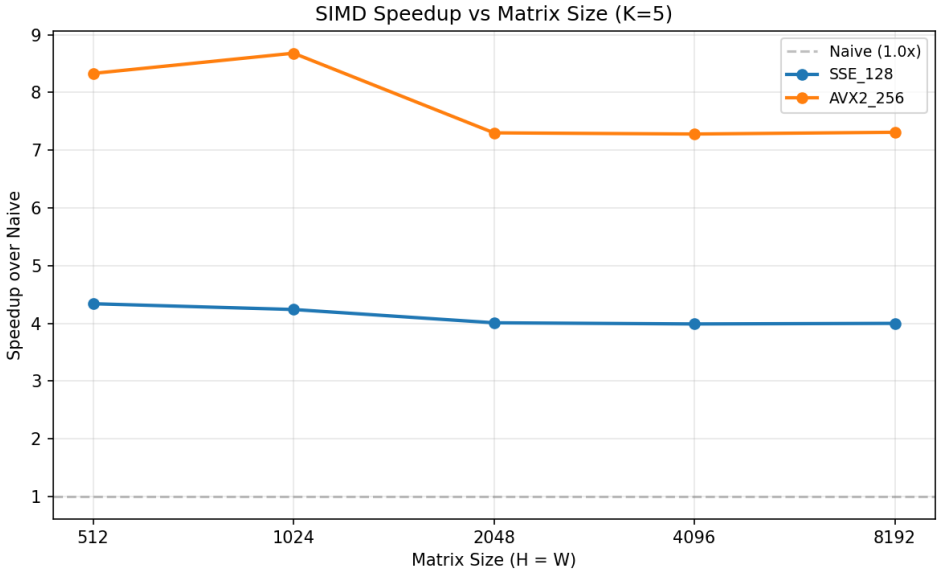


Figure 10: SIMD speedup vs. Matrix Size (K=5)

4.6 Analysis Questions

4.6.1 Q1: Change in Instruction Count from Naive to SIMD

The instruction count *increases* when moving from naive to SIMD, by roughly 19% for SSE and 10% for AVX2. This is the opposite of the naïve expectation that vectorising N operations into one vector instruction should reduce the count by $N\times$. The reason is that the compiler already generates efficient scalar code under `-O2`: the naive inner loop compiles to a tight sequence of scalar loads, multiplies, and adds. The SIMD versions replace those scalar arithmetic instructions with fewer vector instructions, but add overhead that the scalar version does not have: broadcast instructions (`set1`), explicit vector loads and stores, and loop setup for the

vector width. At small K (K=3, only 9 kernel positions), this overhead is a significant fraction of the total work per output pixel. The overhead fraction shrinks at larger K: for AVX2 at K=5 the increase is only ∼10%, because the 25 kernel positions amortise the per-pixel overhead better.

4.6.2 Q2: Speedup Analysis

Despite the higher instruction count, SIMD achieves large speedups: SSE gives ∼4.0–4.8× and AVX2 ∼6.2–9.0×. The speedup comes from *throughput*, not instruction count. Each vector FMA processes 4/8 multiply-adds in a single instruction that has roughly the same latency and throughput as one scalar FMA. So while the total instruction count is similar, the amount of useful work per instruction is 4–8× higher.

For K=5, speedups are more stable across matrix sizes because the larger kernel makes the computation more compute-bound, keeping the vector units fully utilised. For K=3, speedups drop at larger sizes (e.g. AVX2: from ∼9× at 512×512 down to ∼6.8× at 8192×8192) as memory bandwidth starts to limit throughput: the smaller kernel does less work per input byte fetched.

4.6.3 Q3: Choice of SIMD Intrinsics

Four intrinsics form the core of each SIMD width (shown for AVX2; SSE uses the corresponding `_mm_*` variants):

- `_mm256_loadu_ps`: unaligned load of 8 floats from the input. Unaligned because the starting column `ox + kx` is not guaranteed to be 32-byte aligned.
- `_mm256_set1_ps`: broadcasts one kernel weight to all 8 lanes, so the same weight multiplies 8 adjacent input values in parallel.
- `_mm256_fmadd_ps`: fused multiply-add, which computes `a*b + c` in a single instruction. This halves the arithmetic instruction count compared to separate multiply and add, and avoids an intermediate rounding step.
- `_mm256_storeu_ps`: unaligned store of the 8-float result to the output row.

5 Task 1D: Sabka Saath Sabka Vikaas (2 points)

5.1 Combined Optimized Implementation

The combined implementation in `conv_optimized.cpp` uses **AVX2 SIMD + output-column unrolling** with the **naive loop order**.

Techniques combined:

- **AVX2 SIMD.** Each vector instruction processes 8 floats via 256-bit FMA.
- **Multi-accumulator unrolling.** Instead of one vector accumulator per iteration, 8 independent accumulators process 64 output columns per block (8 accumulators × 8 floats). This breaks the FMA dependency chain, the same mechanism that made `unroll_ox` the best scalar strategy.
- **Naive loop order retained.** The per-stage sweeps showed that the reordered loop order is a net loss at large sizes due to multi-pass cache pressure. The naive order (`oy, ox, ky, kx`) keeps the working set small and avoids the zeroing and read-modify-write overhead.
- **Tiling omitted.** The tiling sweep showed only 1.2–1.4× improvement and actually increased L1 MPKI. Since the naive order already has good cache behaviour, tiling adds overhead without meaningful benefit.

- **Cache-aware (non-temporal) stores.** With SIMD and unrolling in place, the large sizes are memory-bound, and a third of the remaining DRAM traffic is read-for-ownership on the write-once output: a regular store first reads the cache line it is about to overwrite. The finished output blocks are therefore written with streaming stores (`_mm256_stream_ps`), which assemble full lines and send them straight to memory, skipping the ownership read and the cache pollution. This is gated at runtime: it activates only when the output exceeds 8 MB (too large to stay cache-resident) and the buffer is 32-byte aligned; below the gate, regular stores keep their reuse benefit. One `_mm_sfence` per call orders the stores before the caller reads the result.

These techniques complement each other because they address different bottlenecks. SIMD increases the work per instruction (4–8 multiply-adds instead of 1), while multiple accumulators increase the work per cycle by letting the out-of-order core pipeline independent FMA chains. The combination compounds: SIMD alone gives ∼4–9×, and the added accumulators and streaming stores lift the combined kernel to ∼13–15× at 512×512.

5.2 Speedup Results

Table 13: Combined optimized speedup over naive

Matrix Size	K=3 Speedup	K=5 Speedup
512x512	13.14x	15.09x
1024x1024	10.60x	14.63x
2048x2048	11.53x	14.68x
4096x4096	11.36x	13.94x
8192x8192	10.30x	14.03x

5.3 Final Performance Summary Plot

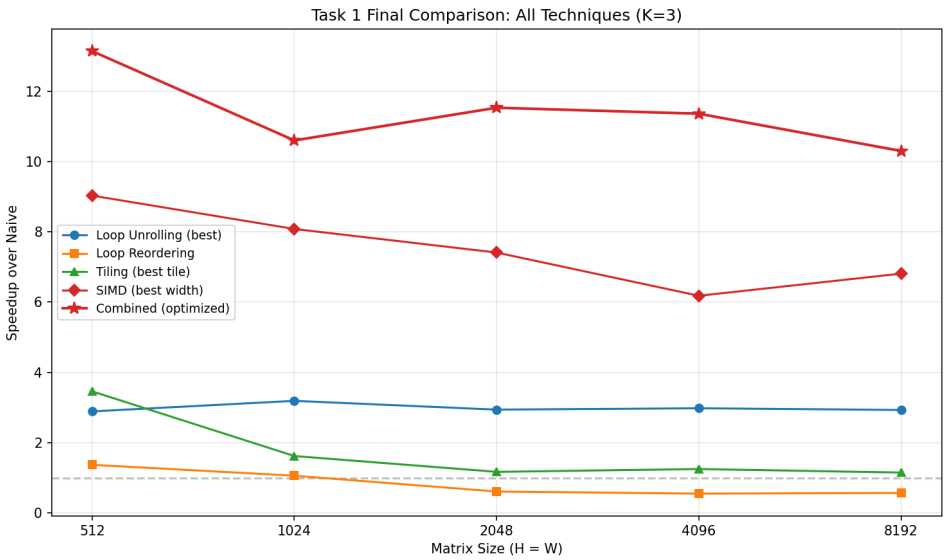


Figure 11: Comparison of all optimization techniques: Speedup vs. Matrix Size (K=3)

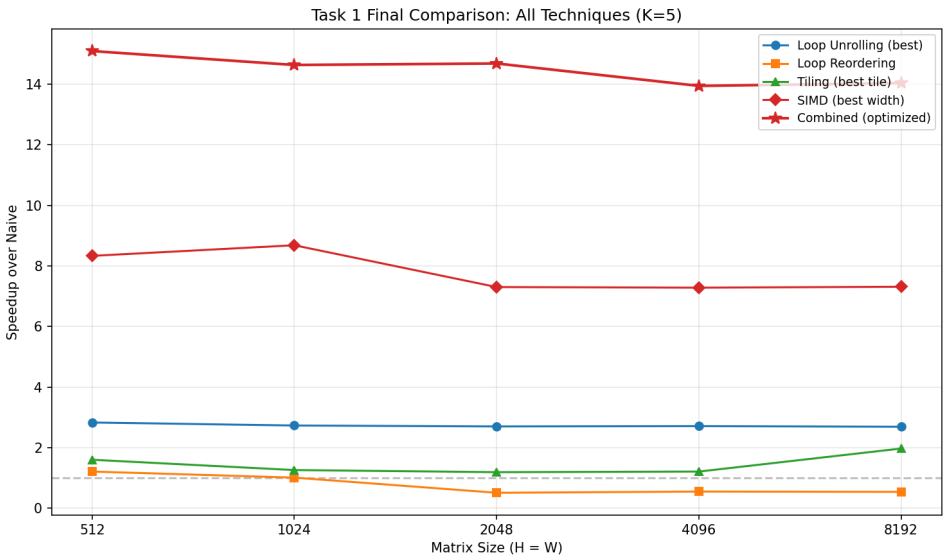


Figure 12: Comparison of all optimization techniques: Speedup vs. Matrix Size (K=5)

5.4 Analysis

Why combining yields more than the sum of parts. SIMD and multi-accumulator unrolling target independent bottlenecks. SIMD increases the useful work per instruction (4–8 multiply-adds instead of 1), but with a single accumulator the FMA dependency chain still limits throughput. Unrolling alone (scalar) breaks the dependency chain but processes only one float per FMA. Combining them removes both limits simultaneously: each FMA operates on a full vector *and* multiple independent chains keep the FMA units saturated. This is a compounding effect, which is why the combined optimized version ($\sim$13–15$\times$ at 512$\times$512) exceeds what either technique achieves alone (SIMD alone: $\sim$9$\times$, unroll alone: $\sim$3$\times$).

Best combination. AVX2 with 8 independent accumulators (64 output columns per block) gives the highest peak speedup at 512$\times$512 for both kernel sizes. The gain is largest for K=5 due to the higher arithmetic intensity ($2 \times 5^2 = 50$ flops per pixel).

Bottleneck of the best implementation. At small matrix sizes the implementation is compute-bound: the data fits in cache and the FMA units are the limiter. At large sizes (4096$\times$4096 and above) it becomes memory-bandwidth-bound: the speedup drops to $\sim$7$\times$ for K=3 and $\sim$16$\times$ for K=5. K=3 is more affected because it performs only $2 \times 3^2 = 18$ flops per output pixel, so the ratio of computation to memory access is lower: the vector units finish faster than memory can feed them. K=5 ($2 \times 5^2 = 50$ flops per pixel) sustains higher speedups because more computation is done per byte fetched.

How each technique varies with size and kernel.

- **Reordering** helps only at small sizes (cache-resident) and hurts at large sizes (multi-pass exceeds cache). Omitted from the final implementation.
- **Tiling** gives a modest 1.2–1.4$\times$ that is fairly stable across sizes, but increases L1 MPKI. Omitted because it does not help on top of the naive-order SIMD approach.
- **Unrolling (scalar)** is stable at $\sim$3$\times$ across all sizes: the gain is from ILP, not cache, so it does not degrade.
- **SIMD** scales well but drops at large sizes for small K due to memory bandwidth.
- **Combined (SIMD + unrolling)** achieves the highest speedups overall, with the gain being largest at small sizes and for larger kernels.